

Valori e tipi

	Tipo	Esempio	Note
Interi	int	3	
Reali	float	3.14	Virgola indicata come punto
Stringhe	str	"ACGT"	Delimitate da " o '. In caso di doppi apici, si possono usare apici all'interno, e viceversa
Liste	list	[0, 1, 2, 3]	Delimitate da [], elementi separati da ,
Tuple	tuple	(0, 1, 2)	Delimitate da (), elementi separati da ,
Dizionari	dict	{"A": "Adenosine", "T": "Thymine"}	Delimitate da {}, coppie separate da , e definite con :
Insiemi	set	{0, 1, 2}	Delimitati da {}, elementi separati da ,

Operatori

	Esempi	Note
Aritmetici	+, -, %, *, /	Seguono le precedenze aritmetiche
Confronto	!=, >, <, >=, <=	
Aritmetici contratti	+=, -=, *=	Inglobano assegnamento, e.g. x += 1 equivale a x = x + 1
Assegnamento	=	Ha precedenza minima
Booleani	and, or, not	Precedenza come indicata
Accesso	[0], [3:5], [-3:]	Possono accedere a elementi singoli o intervalli. Possibile indicizzazione dal termine della collezione

Costrutti

```
for element in collection:
    ...

while condition:
    ...

if condition:
    ...
else:
    ...
```

I costrutti for in e while permettono la ripetizione dei propri corpi: for su tutti gli elementi di una collezione, while fintanto che una condizione è vera. if permette invece di scegliere di eseguire o meno un corpo di istruzioni a seconda della verità della sua condizione. Opzionalmente, else permette di eseguire un corpo alternativo.

Funzioni e metodi

```
def NOME_FUNZIONE(PARAMETRO_1: TIPO_PARAMETRO_1 = VALORE_1) -> TIPO_RITORNO:
    corpo_funzione
```

Dichiara una funzione NOME_FUNZIONE, con un parametro (PARAMETRO_1 che ha valore di default VALORE_1) di tipo TIPO_PARAMETRO_1, e con tipo di ritorno TIPO_RITORNO. Nella definizione, sia tipi che parametri sono opzionali, così come i valori di default dei parametri. Nota che un parametro senza valore di default non può seguire un parametro senza.

Invocazione funzioni.

Sia per ordine che per nome:

```
funzione(parametro_1, ...)
funzione(parametro_1=valore_1, ...)
```

Nota: non si possono indicare parametri per ordine dopo averne indicato uno per nome, e.g.

```
funzione(parametro_1=valore_1, valore_2)
```



risulta in un errore, mentre

```
funzione(valore_1, parametro_2=valore_2)
```



no.

Invocazione metodi.

Equivalente a funzioni, ma accedute dall'oggetto:

```
oggetto.metodo(parametro_1, ...)
```



Classi

```
class Batterio:
    def __init__(self, parametro_1, ...):
        ...
```



Definisce una classe `Batterio`, e fornisce un metodo *costruttore*.

Eccezioni

```
try:
    function_raising_an_exception()
except ValueError:
    ...
finally:
    ...
```



Il corpo del costrutto `try` esegue il proprio corpo. Nel caso in cui un'eccezione venisse sollevata da un'istruzione, tutte le restanti istruzioni del corpo vengono saltate, e il codice passa al costrutto `except`. `except` esegue il proprio corpo fino a terminazione (sempre che non vengano sollevate altre eccezioni). Infine, il costrutto `finally` esegue il proprio corpo, sia che un'eccezione sia stata sollevata nella `try`, sia che non lo sia stata.